

LAPORAN PRAKTIKUM PBO

Keamanan & Integritas Data (Enkapsulasi)

Disusun Oleh:

Endy Pardilian 2511531017

Dosen Pengampu:

NURFIAH, S.ST, M.Kom

Asisten Pratikum:

Ezza Addini



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

PADANG

2026

Langkah Pengerjaan:

A. Praktikum

1. Class Rekening

```
Package pekan3;
Import java.util.ArrayList;
Public class Rekening {
    String nomorRekening;
    String namaPemilik;
    Double saldo;
    Private String pin;

    ArrayList<Transaksi> riwayatTransaksi;

    Public static boolean isValidPin(String pin) {
        If (pin == null || pin.length() != 6) return false;
        For (int i = 0; i < pin.length(); i++) {
            If (!Character.isDigit(pin.charAt(i))) return false;
        }

        For (int i = 0; i < pin.length(); i++) {
            For (int j = i + 1; j < pin.length(); j++) {
                If (pin.charAt(i) == pin.charAt(j)) {
                    Return false;
                }
            }
        }

        For (int i = 0; i < pin.length() - 1; i++) {
            Int selisih = Math.abs(pin.charAt(i) - pin.charAt(i
+ 1));
            If (selisih == 1) {
                Return false;
            }
        }

        Return true;
    }

    Public Rekening (String nomor, String nama, double saldoAwal,
String pinAwal) {
        This.nomorRekening = nomor;
        This.namaPemilik = nama;
        This.saldo = saldoAwal;

        If (isValidPin(pinAwal)) {
            This.pin = pinAwal;
        }else {
            System.out.println("Peringatan: PIN tidak valid!
Menggunakan PIN default 804927");
            This.pin = "804927";
        }

        This.riwayatTransaksi = new ArrayList<>();
    }
}
```

```

        System.out.println("Rekening atas nama "+ namaPemilik + "
berhasil dibuat.");
    }

    Public String getNomorRekening() { return nomorRekening;}
    Public String getNamaPemilik() { return namaPemilik;}

    Public boolean otentikasi(String inputPin) {
        Return this.pin.equals(inputPin);
    }

    Public void setorTunai(double nominal) {
        If (nominal > 0 && nominal <= 500000 ) {
            Saldo += nominal;
            String idTrx = "TRX-S-" +
System.currentTimeMillis();
            Transaksi trxBaru = new Transaksi (idTrx, "Kredit",
nominal);
            riwayatTransaksi.add(trxBaru);

            System.out.println("Setor tunai Rp" + nominal + "
berhasil. Saldo saat ini: Rp" + saldo);
        }else {
            System.out.println("Gagal: nominal setor harus lebih
dari 0 atau melebihi 500.000!");
        }
    }

    Public void tarikTunai (double nominalDebit) {

        If (nominalDebit > 10000) {
            If(saldo < nominalDebit) {
                System.out.println("saldo anda tidak cukup");

            }else {
                Saldo -= nominalDebit;
                String idTrx = "TRX-S-" +
System.currentTimeMillis();
                Transaksi trxBaru = new Transaksi (idTrx, "Debit",
nominalDebit);
                riwayatTransaksi.add(trxBaru);
                System.out.println("tarik tunai Rp"+ nominalDebit +
"berhasil. Saldo saat ini : Rp"+saldo);
            }

        }else {
            System.out.println("Gagal: Nominal tarik harus lebih
dari Rp10000!");
        }
    }

    Public void cetakMutasi() {
        If (riwayatTransaksi.isEmpty()) {
            System.out.println(" Belum ada transaksi");
        }else {
            For (Transaksi trx : riwayatTransaksi) {
                Trx.cetakDetail();
            }
        }
    }

```

```

    }

    }

}

Public void cekInformasi() {
    System.out.println("--- INFO REKENING ---");
    System.out.println("No. Rekening: "+ nomorRekening);
    System.out.println("Nama Pemilik: "+ namaPemilik);
    System.out.println("Saldo Akhir: Rp"+ saldo);
    System.out.println("-----");
    System.out.println("");
}

}

```

Class Rekening tetap berfungsi sebagai blueprint sebuah rekening bank, namun pada versi ini ditambah atribut PIN yang bersifat private dan daftar riwayat transaksi.

a. public static boolean isValidPin

```

public static boolean isValidPin(String pin) {
    if (pin == null || pin.length() != 6) return false;
    for (int i = 0; i < pin.length(); i++) {
        if (!Character.isDigit(pin.charAt(i))) return false;
    }

    for (int i = 0; i < pin.length(); i++) {
        for (int j = i + 1; j < pin.length(); j++) {
            if (pin.charAt(i) == pin.charAt(j)) {
                return false;
            }
        }
    }

    for (int i = 0; i < pin.length() - 1; i++) {
        int selisih = Math.abs(pin.charAt(i) - pin.charAt(i
+ 1));
        if (selisih == 1) {
            return false;
        }
    }

    return true;
}

```

Method ini memeriksa apakah sebuah PIN memenuhi aturan keamanan dan mengembalikan true jika valid atau false jika tidak. Pemeriksaan dilakukan dalam tiga tahap secara berurutan:

1. Panjang dan jenis karakter: PIN tidak boleh null, harus tepat 6 karakter, dan setiap karakternya harus angka (dicek dengan `Character.isDigit`).
2. Tidak ada angka berulang: perulangan bersarang membandingkan setiap karakter pada posisi `I` dengan seluruh karakter di posisi setelahnya (`j = I + 1`). Jika ada dua karakter yang sama, PIN ditolak.
3. Tidak ada angka berurutan: setiap dua karakter yang bersebelahan dihitung selisih nilai absolutnya. Jika selisihnya 1 (misalnya 1 dan 2, atau 5 dan 4), PIN ditolak.

b. `public Rekening`

```
public Rekening (String nomor, String nama, double saldoAwal, String
pinAwal) {
    this.nomorRekening = nomor;
    this.namaPemilik = nama;
    this.saldo = saldoAwal;

    if (isValidPin(pinAwal)) {
        this.pin = pinAwal;
    }else {
        System.out.println("Peringatan: PIN tidak valid!
Menggunakan PIN default 804927");
        This.pin = "804927";
    }

    This.riwayatTransaksi = new ArrayList<>();

    System.out.println("Rekening atas nama "+ namaPemilik + "
berhasil dibuat.");
}
```

Constructor kini menerima empat parameter, yaitu nomor rekening, nama pemilik, saldo awal, dan PIN awal. Setelah menyimpan tiga data pertama, constructor memvalidasi PIN dengan `isValidPin()`. Jika valid, PIN disimpan; jika tidak, sistem memberikan peringatan dan memakai PIN default 804927 sebagai pengaman agar objek tidak pernah berada dalam kondisi tanpa PIN. Constructor juga membuat `ArrayList` kosong untuk riwayatTransaksi karena rekening yang baru dibuat belum memiliki transaksi. Berbeda dengan versi pekan pertama, pesan konfirmasi sekarang tidak lagi menampilkan saldo.

c. `public void setorTunai`

```
public void setorTunai(double nominal) {
    if (nominal > 0 && nominal <= 500000 ) {
        saldo += nominal;
        String idTrx = "TRX-S-" +
System.currentTimeMillis();
        Transaksi trxBaru = new Transaksi (idTrx, "Kredit",
nominal);
        riwayatTransaksi.add(trxBaru);
    }
}
```

```

        System.out.println("Setor tunai Rp" + nominal + "
berhasil. Saldo saat ini: Rp" + saldo);
    }else {
        System.out.println("Gagal: nominal setor harus lebih
dari 0 atau melebihi 500.000!");
    }
}

```

Method ini memproses penyetoran uang. Setoran diterima jika nominalnya lebih dari 0 dan tidak melebihi Rp500.000. Jika lolos validasi, saldo ditambah, lalu dibuat ID transaksi unik dari awalan "TRX-S-" digabung waktu saat ini dalam milidetik (System.currentTimeMillis()). Selanjutnya dibuat objek Transaksi berjenis "Kredit" dan ditambahkan ke riwayatTransaksi sebelum pesan keberhasilan dicetak. Jika validasi gagal, saldo dan riwayat tidak berubah sama sekali dan hanya pesan kegagalan yang ditampilkan.

d. public void tarikTunai

```

public void tarikTunai (double nominalDebit) {
    if (nominalDebit > 10000) {
        if(saldo < nominalDebit) {
            System.out.println("saldo anda tidak cukup");
        }else {
            Saldo -= nominalDebit;
            String idTrx = "TRX-S-" +
System.currentTimeMillis();
            Transaksi trxBaru = new Transaksi (idTrx, "Debit",
nominalDebit);
            riwayatTransaksi.add(trxBaru);
            System.out.println("tarik tunai Rp"+ nominalDebit +
"berhasil. Saldo saat ini : Rp"+saldo);
        }
    }else {
        System.out.println("Gagal: Nominal tarik harus lebih
dari Rp10000!");
    }
}

```

Method ini memproses penarikan uang dengan dua lapis validasi. Lapis pertama memastikan nominal penarikan lebih dari Rp10.000. Lapis kedua memastikan saldo mencukupi; bila saldo lebih kecil daripada nominal, penarikan ditolak dengan pesan "saldo anda tidak cukup". Hanya jika kedua syarat terpenuhi, saldo dikurangi, dibuat objek Transaksi berjenis "Debit", dan objek tersebut disimpan ke riwayatTransaksi. Dengan struktur ini saldo tidak akan pernah bernilai negatif.

e. Public void cetakMutasi

```

f. Public void cetakMutasi() {
g.     if (riwayatTransaksi.isEmpty()) {
h.         System.out.println(" Belum ada transaksi");

```

```

l.         }else {
j.             for (Transaksi trx : riwayatTransaksi) {
k.                 trx.cetakDetail();
l.             }
m.         }
n.     }
o. }

```

Method ini menampilkan seluruh riwayat transaksi rekening. Jika riwayatTransaksi kosong, ditampilkan pesan bahwa belum ada transaksi. Jika berisi, program melakukan perulangan for-each pada setiap objek Transaksi dan memanggil cetakDetail() milik objek tersebut. Ini adalah contoh interaksi antarobjek: Rekening tidak mencetak data transaksi sendiri, melainkan meminta tiap Transaksi mencetak rinciannya.

2. Class Main

```
Package pekan3;
Import java.util.Scanner;
Import java.util.ArrayList;
Public class Main {
    Public static void main (String[]args) {
        Scanner input = new Scanner (System.in);
        ArrayList<Rekening> daftarRekening = new ArrayList<>();
        Rekening akunAktif = null;
        Boolean isRunning = true;

        System.out.println("=== SISTEM PERBANKAN MINI ===");

        While (isRunning) {
            System.out.println("\nMenu Utama: ");
            System.out.println("1. Buka Rekening Baru");
            System.out.println("2. Setor Tunai");
            System.out.println("3. Tarik Tunai");
            System.out.println("4. Cek Informasi Rekening");
            System.out.println("5. Ganti akun aktif");
            System.out.println("6. Cetak mutasi (riwayat)");
            System.out.println("0. Keluar");
            System.out.println("Pilih Menu: ");

            Int pilihan = input.nextInt();
            Input.nextLine();
            Switch (pilihan) {

                Case 1 :
                    System.out.println("Masukkan No Rekening: ");
                    String no = input.nextLine();
                    System.out.println("Masukkan Nama Pemilik: ");
                    String nama = input.nextLine();
                    System.out.println("Masukkan Saldo Awal: ");
                    Double saldo = input.nextDouble();
```

```

        Input.nextLine();
        System.out.println("Masukkan PIN (6 digit angka): ");
        String pin = input.nextLine();
        If(saldo < 50000) {
            System.out.println("saldo awal minimal
Rp50.000");

        }else {
            String pin1 = "";
            Boolean pinValid = false;

            While(!pinValid) {
                System.out.print("Masukkan PIN (6
digit, tanpa angka berulang/berurutan): ");
                Pin1 = input.nextLine();

                If(Rekening.isValidPin(pin1)) {
                    pinValid = true;
                } else {
                    System.out.println("PIN tidak
valid! Syarat: 6 digit angka, tidak ada angka berulang, dan tidak ada
dua angka yang berurutan (misal: 12, 54). Silakan coba lagi.\n");
                }
            }

            akunAktif = new Rekening(no, nama, saldo,pin1);
            daftarRekening.add(akunAktif);
        }
        Break;

        Case 2:
        If (akunAktif == null) {
            System.out.println("Error : Mohon Maaf,
Anda belum Memiliki nomor rekening!");
        }else {
            System.out.println("Masukkan nominal
setor: ");

            Double setor = input.nextDouble();

            If(setor < 10000) {
                System.out.println("saldo setor
minimal Rp10.000");

            }else {

                akunAktif.setorTunai(setor);
            }
        }
        Break;

        Case 3:
        If (akunAktif == null) {
            System.out.println("Error: Anda belum
membuka rekening!");
        }else {
            System.out.print("Masukkan PIN untuk otentikasi: ");

```

```

        String pinInput = input.nextLine();

        If (akunAktif.otentikasi(pinInput)) {
            System.out.println("Masukkan nominal tarik
tunai: ");

            Double tarik = input.nextDouble();
            Input.nextLine();
            akunAktif.tarikTunai(tarik);
        } else {
            System.out.println("Akses Ditolak: PIN yang Anda
masukkan salah!");
        }
    }
    Break;

    Case 4:
        If (akunAktif == null) {
            System.out.println("Error: Anda belum
membuka rekening!");
        } else {
            akunAktif.cekInformasi();
        }
        Break;

    Case 5:
        If (daftarRekening.isEmpty()) {
            System.out.println("Belum ada rekening
yang terdaftar!");
        } else {
            System.out.println("DAFTAR REKENING
TERDAFTAR ");

            For (int i = 0; i <
daftarRekening.size(); i++) {
                System.out.println("Pilihan Ke-"
+ (i + 1) + ":");

                daftarRekening.get(i).cekInformasi();

                System.out.println("=====");
            }

            System.out.print("Pilih nomor indeks
rekening yang ingin diaktifkan (1 - " + daftarRekening.size() + "): ");
            Int indeksPilihan = input.nextInt();

            If (indeksPilihan >= 1 && indeksPilihan
<= daftarRekening.size()) {

                akunAktif =
daftarRekening.get(indeksPilihan - 1);
                System.out.println("Berhasil
berganti ke akun pilihan Anda!");
            } else {
                System.out.println("Nomor
pilihan tidak valid!");
            }
        }
    }
}

```

```

        }
        Break;

        Case 6:
            If (akunAktif == null) {
                System.out.println("Error: Anda belum
membuka rekening!");
            }else {
                System.out.print("Masukkan PIN untuk otentikasi: ");
                String pinInput = input.nextLine();

                If (akunAktif.otentikasi(pinInput)) {
                    akunAktif.cetakMutasi();
                } else {
                    System.out.println("Akses Ditolak: PIN yang Anda
masukkan salah!");
                }
            }
        Break;

        Case 0:
            isRunning = false;
            System.out.println("Sistem ditutup. Terima
kasih!");

            Break;

        Default:
            System.out.println("Pilihan tidak
valid!");
        }
    }
    Input.close();
}
}

```

Class Main berfungsi sebagai pengendali program dan titik masuk yang menjalankan seluruh alur interaksi dengan pengguna. Class ini bertugas menampilkan menu pilihan di layar, membaca input yang diketik pengguna, menyimpan seluruh rekening yang pernah dibuat ke dalam sebuah daftar (ArrayList), serta memutuskan objek Rekening mana yang sedang aktif digunakan. Setiap kali pengguna memilih suatu menu, misalnya setor atau tarik tunai, Main tidak memproses transaksi itu sendiri, melainkan memanggil method yang sudah tersedia di class Rekening untuk melakukan pekerjaannya.

a. public class Main

```

package pekan1;
import java.util.Scanner;

public class Main1 {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        Rekening akunAktif = null;
        boolean isRunning = true;
    }
}

```

```

System.out.println("=== SISTEM PERBANKAN MINI ===");

while (isRunning) {
    System.out.println("\nMenu Utama:");
    System.out.println("1. Buka Rekening Baru");
    System.out.println("2. Setor Tunai");
    System.out.println("3. Tarik Tunai");
    System.out.println("4. Cek Informasi Rekening");
    System.out.println("0. Keluar");
    System.out.print("Pilih menu: ");

    int pilihan = input.nextInt();
    input.nextLine();

    switch (pilihan) {

```

Method ini adalah titik masuk (entry point) program. Di awal method, dibuat objek Scanner untuk membaca input dari keyboard, variabel `akunAktif` bertipe Rekening yang sengaja diinisialisasi dengan nilai null karena belum ada rekening yang dibuat saat program baru dijalankan, serta variabel boolean `isRunning` yang menjadi saklar agar program terus berjalan dalam perulangan menu sampai pengguna memilih keluar. Perlu diperhatikan bahwa versi ini berbeda dari versi sebelumnya: di sini tidak ada `ArrayList<Rekening>`, sehingga program hanya bisa menangani satu rekening aktif pada satu waktu — setiap kali menu 1 dipilih, rekening lama akan otomatis tertimpa oleh rekening baru.

b. Switch-Case

1. Case 1 Buka Rekening Baru

```

case 1:
    System.out.print("Masukkan No Rekening: ");
    String no = input.nextLine();
    System.out.print("Masukkan Nama Pemilik: ");
    String nama = input.nextLine();
    System.out.print("Masukan Saldo Awal: ");
    double saldo = input.nextDouble();

    akunAktif = new Rekening(no, nama, saldo);
    break;

```

Bagian ini meminta pengguna memasukkan nomor rekening, nama pemilik, dan saldo awal secara berurutan. Setelah semua data terkumpul, program membuat objek Rekening baru dengan memanggil constructor (`new Rekening(no, nama, saldo)`), lalu hasilnya langsung disimpan ke variabel `akunAktif`.

Output:

```

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu: 5
Pilihan tidak valid!

```

```

=== SISTEM PERBANKAN MINI ===
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu: 1
Masukkan No Rekening: 12345
Masukkan Nama Pemilik: Endy
Masukan Saldo Awal: 100000
Rekening atas nama Endy berhasil dibuat dengan saldo Rp100000.0

```

2. Case 2 Setor Tunai

```

case 2:
    if (akunAktif == null) {
        System.out.println("Error: Mohon maaf,
Anda belum memiliki nomor rekening!");
    } else {
        System.out.print("Masukkan nominal
setor: ");
        double setor = input.nextDouble();
        akunAktif.setorTunai(setor);
    }
    break;

```

Program terlebih dahulu memeriksa apakah akunAktif masih null. Jika iya, artinya belum ada rekening yang dibuat sehingga transaksi tidak bisa dilakukan dan pesan error ditampilkan. Jika sudah ada rekening aktif, program meminta nominal setor dari pengguna, lalu memanggil method setorTunai() milik objek akunAktif — seluruh validasi nominal (minimal setor) diproses di dalam class Rekening, bukan di Main.

Output:

```

=== SISTEM PERBANKAN MINI ===
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu: 2
Error: Mohon maaf, Anda belum memiliki nomor rekening!

```

```

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu: 2
Masukkan nominal setor: 10000
Setor tunai Rp10000.0 berhasil. Saldo saat ini: Rp110000.0

```

3. Case 3 Tarik Tunai

```

case 3:
    if (akunAktif == null) {
        System.out.println("Error: Mohon maaf,
Anda belum memiliki nomor rekening!");
    } else {

```

```

                                System.out.println("Masukkan nominal
tarik: ");

                                double tarik = input.nextDouble();
                                akunAktif.tarikTunai(tarik);
                                }
                                break;

```

Alurnya serupa dengan setor tunai: program memastikan ada akun aktif terlebih dahulu, kemudian meminta nominal tarik dari pengguna dan memanggil method `tarikTunai()` milik `akunAktif`. Semua validasi terkait batas minimal dan kecukupan saldo diproses di dalam class `Rekening`, sehingga class `Main` hanya berperan meneruskan input pengguna ke method yang bersangkutan.

Output:

```

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu: 3
Error: Mohon maaf, Anda belum memiliki nomor rekening!

```

```

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu: 3
Masukkan nominal tarik:
9000
Gagal: Minimal tarik 10000!

```

```

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu: 3
Masukkan nominal tarik:
10000
Tarik tunai Rp10000.0 berhasil. Saldo anda saat ini: 10000.0

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu: 3
Masukkan nominal tarik:
11000
Gagal : Saldo anda tidak cukup!

```

4. Case 4 Cek Informasi Rekening

```

case 4:

                                if (akunAktif == null) {
                                    System.out.println("Error: Anda belum
membuka rekening!");
                                } else {
                                    akunAktif.cekInformasi();
                                }
                                break;

```

Program memeriksa apakah akunAktif masih null terlebih dahulu. Jika belum ada rekening yang dibuka, pesan error ditampilkan. Jika sudah ada, method cekInformasi() milik akunAktif dipanggil untuk menampilkan nomor rekening, nama pemilik, dan saldo akhir ke layar.

Output:

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu: 4
Error: Anda belum membuka rekening!
```

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu: 4
--- INFO REKENING ---
No. Rekening : 12345
Nama Pemilik : Endy
Saldo Akhir  : Rp10000.0
-----
```

5. Case 5 Ganti Akun

```
case 5:
    if (daftarRekening.isEmpty()) {
        System.out.println("Error: Belum ada
rekening yang terdaftar!");
    } else {
        System.out.println("Masukkan nomor
rekening yang ingin diaktifkan: ");
        String nomorDicari = input.nextLine();

        Rekening ditemukan = null;
        for (Rekening r : daftarRekening) {
            if
(r.nomorRekening.equalsIgnoreCase(nomorDicari)) {
                ditemukan = r;
                break;
            }
        }

        if (ditemukan != null) {
            akunAktif = ditemukan;
            System.out.println("Berhasil
berpindah ke rekening atas nama " + akunAktif.namaPemilik);
        } else {
            System.out.println("Error: Nomor
rekening '" + nomorDicari + "' tidak ditemukan!");
        }
    }
    break;
```

Bagian ini memungkinkan pengguna berpindah ke rekening lain yang sudah terdaftar. Program pertama-tama memeriksa apakah daftarRekening masih kosong;

jika kosong, pengguna diberi tahu bahwa belum ada rekening yang bisa dipilih. Jika tidak kosong, pengguna diminta memasukkan nomor rekening yang ingin diaktifkan, kemudian program melakukan pencarian di dalam daftarRekening menggunakan perulangan for. Apabila rekening dengan nomor tersebut ditemukan, akunAktif akan diperbarui ke rekening itu disertai pesan konfirmasi; jika tidak ditemukan, pesan error akan ditampilkan kepada pengguna.

Output:

```
=== SISTEM PERBANKAN MINI ===  
  
Menu Utama:  
1. Buka Rekening Baru  
2. Setor Tunai  
3. Tarik Tunai  
4. Cek Informasi Rekening  
5. Ganti Akun  
0. Keluar  
Pilih menu:  
5  
Error: Belum ada rekening yang terdaftar!
```

```
Menu Utama:  
1. Buka Rekening Baru  
2. Setor Tunai  
3. Tarik Tunai  
4. Cek Informasi Rekening  
5. Ganti Akun  
0. Keluar  
Pilih menu:  
1  
Masukan No Rekening:  
12345  
Masukan Nama Pemilik:  
Endy  
Masukan Saldo Awal: 100000  
Rekening atas nama Endy berhasil dibuat dengan saldo Rp100000.0
```

```
Menu Utama:  
1. Buka Rekening Baru  
2. Setor Tunai  
3. Tarik Tunai  
4. Cek Informasi Rekening  
5. Ganti Akun  
0. Keluar  
Pilih menu:  
1  
Masukan No Rekening:  
12345  
Error: Nomor rekening 12345sudah terdaftar!
```

```
Menu Utama:  
1. Buka Rekening Baru  
2. Setor Tunai  
3. Tarik Tunai  
4. Cek Informasi Rekening  
5. Ganti Akun  
0. Keluar  
Pilih menu:  
1  
Masukan No Rekening:  
23456  
Masukan Nama Pemilik:  
Pardilian  
Masukan Saldo Awal: 10000  
Rekening atas nama Pardilian berhasil dibuat dengan saldo Rp10000.0
```

```

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
0. Keluar
Pilih menu:
5
Masukkan nomor rekening yang ingin diaktifkan:
12345
Berhasil berpindah ke rekening atas nama Endy

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
0. Keluar
Pilih menu:
5
Masukkan nomor rekening yang ingin diaktifkan:
23456
Berhasil berpindah ke rekening atas nama Pardilian

```

6. Case 6 Cetak Mutasi

```

Case 6:
        If (akunAktif == null) {
            System.out.println("Error: Anda belum
membuka rekening!");
        }else {
            System.out.print("Masukkan PIN untuk otentikasi: ");
            String pinInput = input.nextLine();

            If (akunAktif.otentikasi(pinInput)) {
                akunAktif.cetakMutasi();
            } else {
                System.out.println("Akses Ditolak: PIN yang Anda
masukkan salah!");
            }
        }
        Break;

```

Menu ini menampilkan riwayat transaksi rekening aktif. Alurnya serupa dengan tarik tunai: program memastikan ada akun aktif, meminta PIN, lalu memanggil cetakMutasi() hanya jika PIN benar. Riwayat transaksi bersifat rahasia sehingga perlu otentikasi terlebih dahulu.

Output:

```

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. ganti akun aktif
6. cetak mutasi (riwayat)
7. ganti pin
0. Keluar
Pilih Menu:
6
ID: TRX-S-1790691604005 | Jenis Debit | Nominal: Rp50000.0

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. ganti akun aktif
6. cetak mutasi (riwayat)
7. ganti pin
0. Keluar
Pilih Menu:

```

7. Case 0 Keluar

```

case 0:
    isRunning = false;
    System.out.println("Sistem ditutup. Terima
    kasih!");
    break;

```

Saat menu ini dipilih, variabel `isRunning` diubah menjadi `false` sehingga perulangan `while` akan berhenti pada pengecekan kondisi berikutnya, disertai pesan penutup sebagai tanda program selesai dijalankan.

Output:

```

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu: 0
Sistem ditutup. Terima kasih!

```

3. Class Transaksi

```
Package pekan3;

Public class Transaksi {
    String idTransaksi;
    String jenis;
    Double nominal;

    Public Transaksi (String id, String jenis, double nominal) {
        This.idTransaksi = id;
        This.jenis = jenis;
        This.nominal = nominal;
    }
    Public void cetakDetail() {
        System.out.println("ID: "+ idTransaksi + " | Jenis: " +
jenis + " | Nominal: Rp" + nominal);
    }
}
```

Class Transaksi merupakan class baru yang berfungsi sebagai model (cetakan data) untuk satu catatan mutasi pada sebuah rekening. Setiap kali terjadi setor tunai atau tarik tunai yang berhasil, satu objek Transaksi dibuat dan disimpan ke dalam riwayat rekening. Class ini memiliki tiga atribut, yaitu idTransaksi sebagai penanda unik transaksi, jenis yang berisi kategori transaksi (“Kredit” untuk uang masuk dan “Debit” untuk uang keluar), serta nominal yang menyimpan jumlah uang yang ditransaksikan.

a. Constructor Transaksi

Constructor menerima tiga parameter dan menyimpannya ke atribut objek menggunakan kata kunci this. Penggunaan this diperlukan karena nama parameter jenis dan nominal sama dengan nama atributnya, sehingga this.jenis = jenis berarti “isi atribut milik objek ini dengan nilai dari parameter”.

b. Public void cetakDetail

Method ini menampilkan satu baris rincian transaksi ke layar dengan format ID, jenis, dan nominal. Method ini hanya membaca data dan tidak mengubah apa pun. Dengan menempatkan method pencetak di dalam class Transaksi itu sendiri, objek Transaksi mampu “menjelaskan dirinya sendiri” sehingga class lain cukup memanggil cetakDetail() tanpa perlu mengetahui isi atributnya.

B. Tugas Praktikum

1. Menu 1 (Buka Rekening): Sebelum menginstansiasi objek baru, minta user menginputkan PIN (String angka 6 digit).

```
public static boolean isValidPin
```

```

public static boolean isValidPin(String pin) {
    if (pin == null || pin.length() != 6) return false;
    for (int I = 0; I < pin.length(); i++) {
        if (!Character.isDigit(pin.charAt(i))) return false;
    }

    For (int I = 0; I < pin.length(); i++) {
        For (int j = I + 1; j < pin.length(); j++) {
            If (pin.charAt(i) == pin.charAt(j)) {
                Return false;
            }
        }
    }

    For (int I = 0; I < pin.length() - 1; i++) {
        Int selisih = Math.abs(pin.charAt(i) - pin.charAt(I
+ 1));
        If (selisih == 1) {
            Return false;
        }
    }

    Return true;
}

```

Method ini memeriksa apakah sebuah PIN memenuhi aturan keamanan dan mengembalikan true jika valid atau false jika tidak. Panjang dan jenis karakter: PIN tidak boleh null, harus tepat 6 karakter, dan setiap karakternya harus angka (dicek dengan Character.isDigit).

Case 1 :

```

System.out.println("Masukkan No Rekening: ");
String no = input.nextLine();
System.out.println("Masukkan Nama Pemilik: ");
String nama = input.nextLine();
System.out.println("Masukkan Saldo Awal: ");
Double saldo = input.nextDouble();
Input.nextLine();
System.out.println("Masukkan PIN (6 digit angka): ");
String pin = input.nextLine();
If(saldo < 50000) {
    System.out.println("saldo awal minimal
Rp50.000");

}else {
    String pin1 = "";
    Boolean pinValid = false;

    While(!pinValid) {
        System.out.print("Masukkan PIN (6
digit, tanpa angka berulang/berurutan): ");
        Pin1 = input.nextLine();

        If(Rekening.isValidPin(pin1)) {
            pinValid = true;
        } else {

```

```

                                System.out.println("PIN tidak
valid! Syarat: 6 digit angka, tidak ada angka berulang, dan tidak ada
dua angka yang berurutan (misal: 12, 54). Silakan coba lagi.\n");
                                }
                                }

                                akunAktif = new Rekening(no, nama, saldo, pin1);
                                daftarRekening.add(akunAktif);
                                }
                                Break;

```

Menu ini meminta nomor rekening, nama pemilik, saldo awal, dan PIN. Program lalu memeriksa saldo awal; jika kurang dari Rp50.000, pembuatan rekening dibatalkan dengan pesan peringatan. Jika memenuhi, program masuk ke perulangan while (!pinValid) yang terus meminta PIN sampai Rekening.isValidPin() mengembalikan true, sehingga PIN yang salah tidak menghentikan program, melainkan pengguna diminta mengulang. Setelah PIN valid, objek Rekening dibuat melalui constructor, langsung dijadikan akunAktif, dan ditambahkan ke daftarRekening. Dengan demikian pembukaan rekening baru tidak lagi menimpa rekening lama.

Output:

```

=== SISTEM PERBANKAN MINI ===

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. ganti akun aktif
6. cetak mutasi (riwayat)
7. ganti pin
0. Keluar
Pilih Menu:
1
Masukkan No PIN awal (6 digit angka): 010101
Masukkan No Rekening: 45678
Masukkan Nama Pemilik: endy
Masukkan Saldo Awal: 2000000
Rekening atas nama endy berhasil dibuat.

```

2. Keamanan Transaksi: Perbarui Menu 3 (Tarik Tunai) dan Menu 6 (Cetak Mutasi). Sebelum Sistem mengeksekusi kedua method tersebut, sistem harus meminta user memasukkan PIN.

- Panggil fungsi akunAktif.otentikasi(pinYangDiinput).
- Jika mengembalikan nilai true, jalankan transaksi.

- Jika false, batalkan dan cetak: “Akses Ditolak: PIN yang Anda masukkan salah!”

Case 3

```
Case 3:
        If (akunAktif == null) {
            System.out.println("Error: Anda belum
membuka rekening!");
        }else {
            System.out.print("Masukkan PIN untuk otentikasi: ");
            String pinInput = input.nextLine();

            If (akunAktif.otentikasi(pinInput)) {
                System.out.println("Masukkan nominal tarik
tunai: ");
                Double tarik = input.nextDouble();
                Input.nextLine();
                akunAktif.tarikTunai(tarik);
            } else {
                System.out.println("Akses Ditolak: PIN yang Anda
masukkan salah!");
            }
        }
        Break;
```

Penarikan uang kini dilindungi PIN. Setelah memastikan ada akun aktif, program meminta PIN dan memverifikasinya dengan `akunAktif.otentikasi(pinInput)`. Jika PIN benar, pengguna diminta memasukkan nominal tarik dan `tarikTunai()` dipanggil; jika salah, ditampilkan pesan “Akses Ditolak” dan proses penarikan tidak dijalankan. Perhatikan bahwa Main tidak pernah membaca PIN asli, hanya menerima hasil true atau false dari method otentikasi.

Output:

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. ganti akun aktif
6. cetak mutasi (riwayat)
7. ganti pin
0. Keluar
Pilih Menu:
3
masukkan pin anda:
010101
Masukkan nominal tarik tunai:
50000
tarik tunai Rp50000.0 berhasil. Saldo saat ini : Rp1950000.0
```

Case 6

```
case 6:
    if (akunAktif == null) {
        System.out.println("Error: Anda belum
membuka rekening!");
    } else {
        System.out.print("Masukkan PIN untuk otentikasi: ");
        String pinInput = input.nextLine();

        If (akunAktif.otentikasi(pinInput)) {
            akunAktif.cetakMutasi();
        } else {
            System.out.println("Akses Ditolak: PIN yang Anda
masukkan salah!");
        }
    }
    Break;
```

Menu ini menampilkan riwayat transaksi rekening aktif. Alurnya serupa dengan tarik tunai: program memastikan ada akun aktif, meminta PIN, lalu memanggil cetakMutasi() hanya jika PIN benar. Riwayat transaksi bersifat rahasia sehingga perlu otentikasi terlebih dahulu.

Output:

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. ganti akun aktif
6. cetak mutasi (riwayat)
7. ganti pin
0. Keluar
Pilih Menu:
6
ID: TRX-S-1790691604005 | Jenis Debit | Nominal: Rp50000.0

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. ganti akun aktif
6. cetak mutasi (riwayat)
7. ganti pin
0. Keluar
Pilih Menu:
```

C. Link Repositori GitHub

https://github.com/endypardilian/PrakPBO_C_2511531017/commit/27229abb25f329e03ea470c60d9b965206211ea3